

Machine Learning 5

Word Representation

Consider a vocabulary

$$V = [a, aaron, \dots, zulu, \text{UNK}_i]$$

where the size of our set is,

$$|V| = 10,000$$

to represent these words we can use **1-hot representation**

1-hot representation

Man (5391)	Woman (1851)	King (4914)	Queen (7157)
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$

Example:

I want a glass of orange juice

→ I want a glass of apple ?

Note: 1-hot struggles to generalise, this because the inner product of any 2 1-hot-vectors is 0. So, the model doesn't learn that apple-orange are more similar then say king-orange or sky-orange.

Note: Better to have a featurised representation than 1-hot-representation

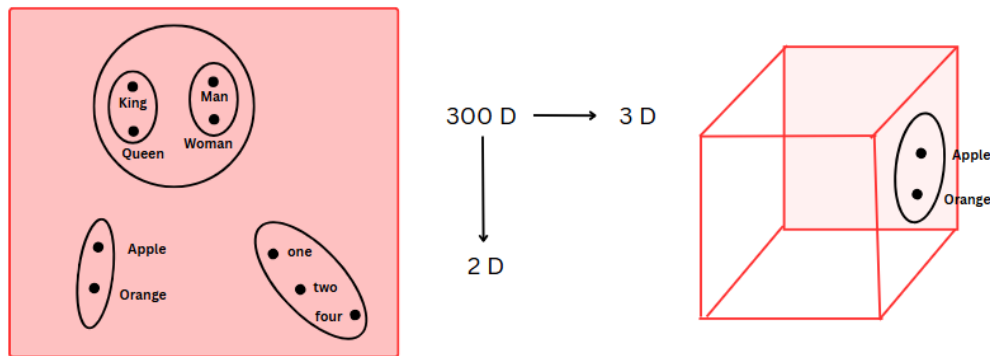
Featurised Representation

	Man (5391)	Woman (9851)	King (4914)	Queen (7157)	...	Apple	Orange	...
Gender	-1	1	-0.95	0.97	...	0.00	0.01	
Royal	0.01	0.02	0.93	0.95	...	-0.01	0.00	
Age	0.03	0.02	0.7	0.69	...	0.03	-0.02	
Food	0.3	0.1	0.02	0.01	...	0.95	0.97	
⋮								
Size								
Cost								
⋮								
	e_{5391}	e_{9853}						

Note: Notice how the scores for orange and apple are similar, therefore apple is more easily associated with juice when orange already is.

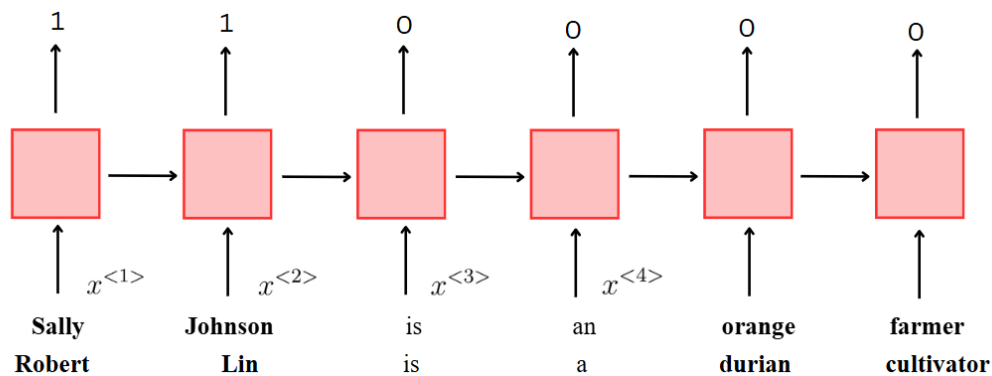
Example: Visualising Word Embeddings in 2D

t-SNE algorithm



Note: Words are related to each other within a concentric context of circles

Example: Named-Entity Recognition



Note: Usually large training sets are used for Named-Entity-Recognition, 1 B - 100 B words. We can then apply transfer learning to a task of Named-entity Recognition on a smaller set.

Note: In actuality, we use BRNNs not RNNs.

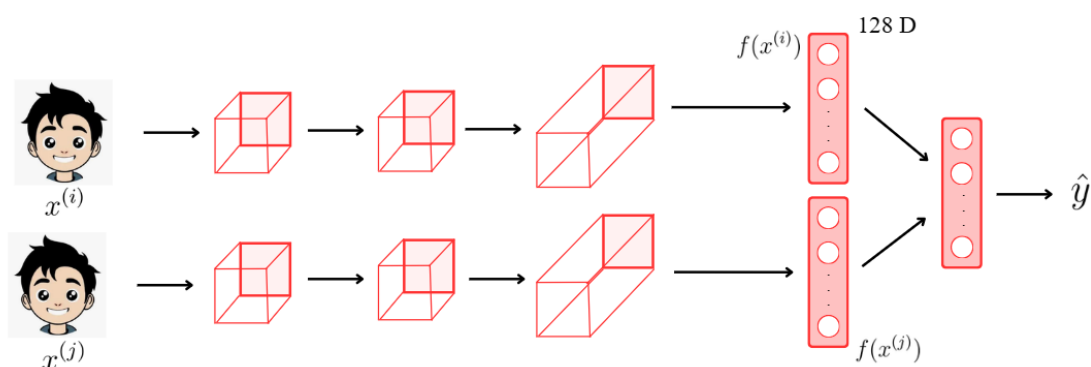
Transfer Learning and Word Embeddings

1. Learn word embeddings from large text corpus (1 - 100 B words)
2. Transfer embedding to a new task with smaller training set
3. Optional: Continue to fine-tune the word embeddings with new data.

Note: Word-Embeddings tend to make the biggest difference when the training data is relatively small.

Example: Relation to face-encoding (like embedding)

Deep face: tell if 2 face-images are of the same person ?



Example:

	Man (5391)	Woman (9851)	King (4914)	Queen (7157)	...	Apple	Orange	...
Gender	-1	1	-0.95	0.97	...	0.00	0.01	
Royal	0.01	0.02	0.93	0.95	...	-0.01	0.00	
Age	0.03	0.02	0.7	0.69	...	0.03	-0.02	
Food	0.3	0.1	0.02	0.01	...	0.95	0.97	
⋮								

if from our previous featurised representation example, we take,

$$e_{man} - e_{woman} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$$

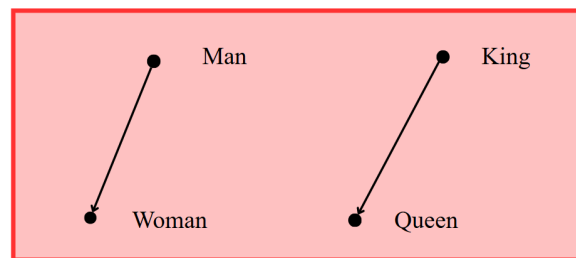
and,

$$e_{king} - e_{queen} = \begin{bmatrix} -2 \\ 0 \\ 0 \\ \vdots \end{bmatrix}$$

Since, these vectors are similar, our model would say that, "man is to woman what king is to queen", i.e., man : woman :: king : queen. Stuff like this can help us with **analogous reasoning**.

"Man is to woman." → "King is to ?."

Analogies using word vectors



300 D

$$e_{man} - e_{woman} \approx e_{king} - e_{?}$$

find,

$$w_{(word)} : \arg \max_w sim(e_w, e_{king} - e_{man} + e_{woman})$$

where sim() is the similarity function,

$$sim(e_w, e_{king} - e_{man} + e_{woman})$$

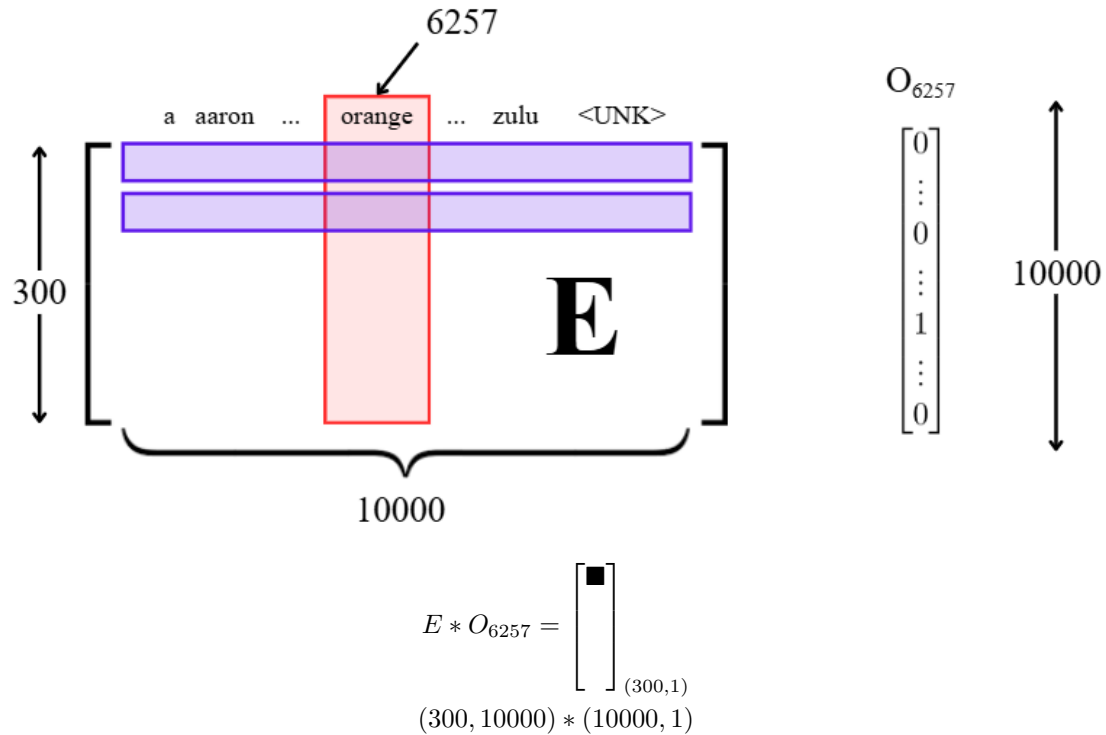
Cosine Similarity

$$\rightarrow sim(u, v) = \frac{u^T v}{||u||_2 ||v||_2}$$

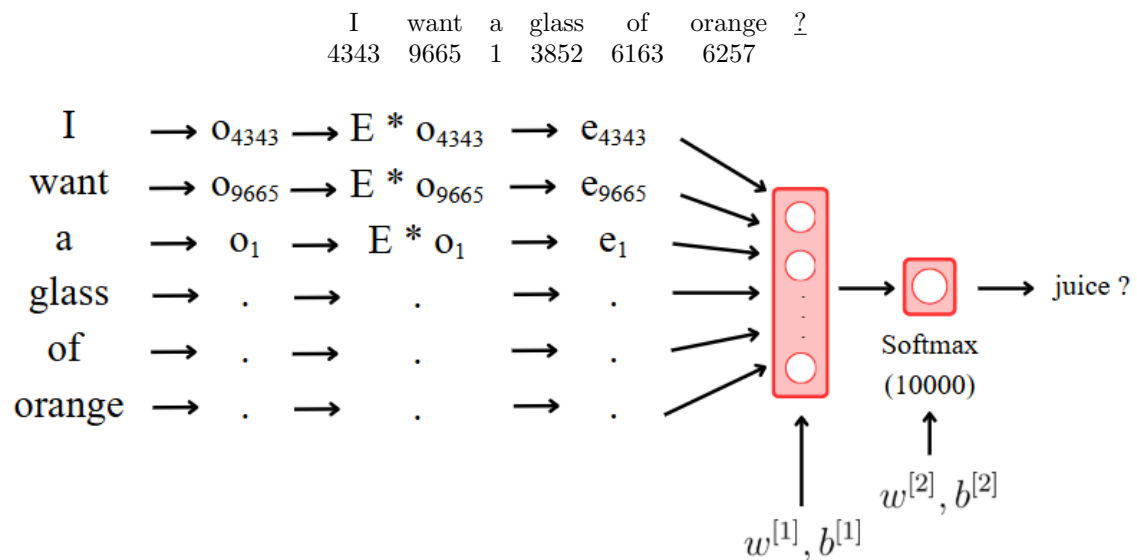
Example:

- Man : Woman :: King : Queen
- Yen : Japan :: Ruble : Russia
- Big : Bigger :: Tall : Taller
- Ottawa : Canada :: Nairobi : Kenya

Embedding Matrix (E)



Neural language model



Note: Each vector e_x is of length 1800 ($6 * 300$), where the 6 comes from the number of words stacked together and 300 comes from the height of the embedding matrix.

Note: 1800 can be dropped to 1200 if you only predict very last 4 words → hyper-parameter testing

Other Context/Target Pairs

Example:

I want a glass of orange juice to go along with my cereal.

Depending on how we choose the **context**, we get different learning problems:

- **Last 4 words:** context = {a, glass, of, orange}, target = juice (the language-model approach used above)
- **4 words on left & right:** context = {a, glass, of, orange, to, go, along, with}, target = juice
- **Last 1 word:** context = {orange}, target = juice (simpler, different learning problem)
- **Nearby 1 word:** context = {glass} (or any nearby word), target = juice

(this is the idea behind **skip-gram**)

Note: Changing how context/target pairs are constructed changes what the algorithm learns to predict this choice is what distinguishes **language models**, CBOW (**continuous bag of words**), and **skip-gram** from each other.

Word2Vec Skip-grams

Note: Instead of always having the preceding words as context we can come up with context-target pairs.

Example:

context	target
orange	juice
orange	glass
orange	my
↑	↑

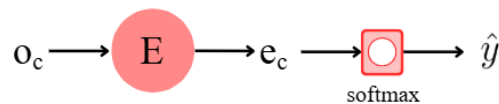
Skip-Gram-Model (as a supervised problem)

Vocabulary size = 10000

We can have the following setup,

$$x \longrightarrow y$$
$$\text{Context, } c \left(\begin{array}{c} \text{"orange"} \\ \text{6257} \end{array} \right) \rightarrow \text{Target, } t \left(\begin{array}{c} \text{"juice"} \\ \text{4834} \end{array} \right)$$

Which we can abstract as,



$$e_c = E o_c$$

where the softmax is,

$$\text{softmax} : p(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^{10000} e^{\theta_j^T e_c}}$$

where θ_t = parameter associated with output t.

$$\mathcal{L}(\hat{y}, y) = - \sum_{i=1}^{10000} y_i \log \hat{y}_i$$

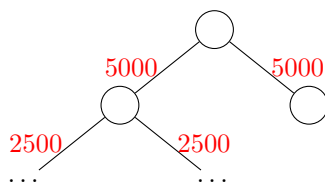
where $y = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \leftarrow 4834$

Problem with Softmax Classification

$$p(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^{10000} e^{\theta_j^T e_c}}$$

Note: The denominator sum is a massive computational undertaking for each repetition as it's performing a multiplicative sum over 10,000 samples.

Solution - 1



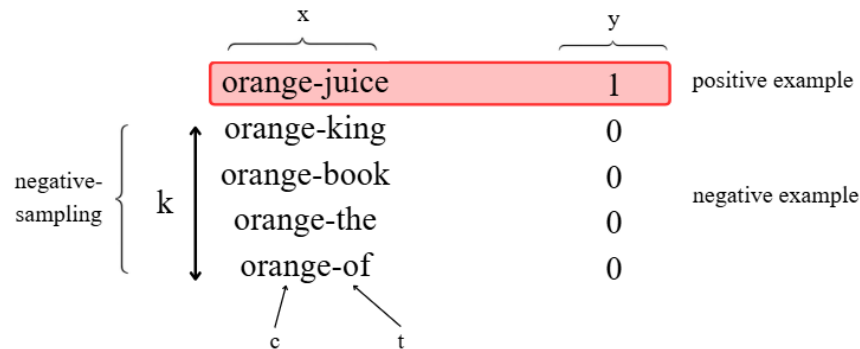
Note: Hierarchical Softmax: In real applications it doesn't use a balanced tree all the time, it could be used in such a way that more common words were closer to the top.

Note: Using a tree we can achieve $O(\log|v|)$ lookup instead of $O(|v|)$. Splitting our vocabulary at each node, until we arrive at the word.

How to sample context, c

Defining a new learning problem,

"I want a glass of orange juice to go along with my cereal."

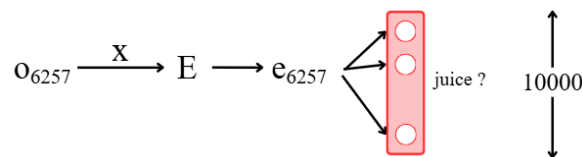


- $K = 2-5$: larger dataset
- $K = 5-20$: smaller dataset

We can define the following model in the softmax classification model,

$$P(y = 1|c, t) = \sigma(\theta_t^T e_c)$$

orange (6257),



Note: So, instead of one giant classification problem, we have instead created 10,000 little (cheap) classification problems.

Selecting negative examples

1. We could sample based on empirical-frequency of words in our corpus $P(w_i)$
 - But this will lead to a very high concentration of words like "the", "and", "a".
2. If we sample uniformly randomly $\frac{1}{|v|}$
 - We still get a very non-representative set of words.

Note: Empirically what will work the best is to take the heuristic value,

$$P(w_i) = \frac{f(w_i)^{\frac{3}{4}}}{\sum_{j=1}^{10,000} f(w_j)^{\frac{3}{4}}}$$

GloVe (Global Vectors for Word Representation)

"I want a glass of orange juice to get alone with my cereal."

Given c and t,

$X_{i,j}$ = "Number of times j (t) appears in context of i (c)".

Note: we might have $X_{i,j} = X_{j,i}$

Model

minimise $\sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(X_{i,j})(\theta_i^T e_j + b_i + b_j - \log X_{i,j})^2$

here, $f(X_{i,j})$ is a weighting term,

$f(X_{i,j}) = 0$ if $n_{i,j} = 0$.

θ_i, e_j are also symmetric,

$e_w^{(final)} = \frac{e_w + \theta_w}{2}$

we also set the convention, "log 0 = 0".

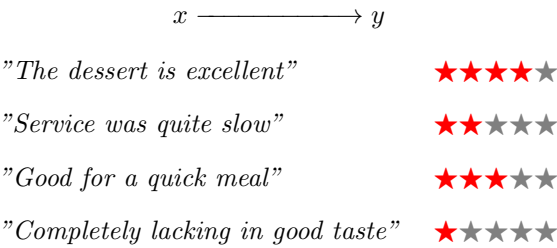
Note: A problem is that if there was an invertible matrix A, then,

$\theta_i^T e_j = (A\theta_i)^T (A^{-T} e_j) = \theta_i^T e_j = \theta_i^T e_j$

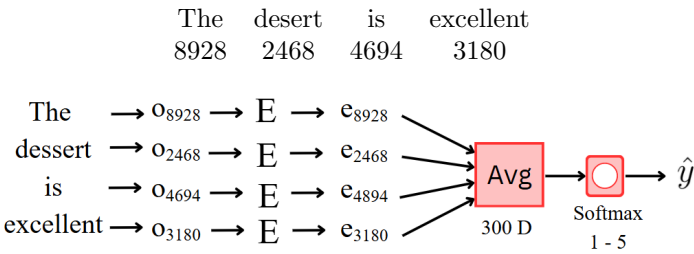
which shows that with an algorithm like this you can guarantee that the axes used to represent the features would be well-adjusted with what might be humanly interpretable.

Sentiment Classification Problem

Example:



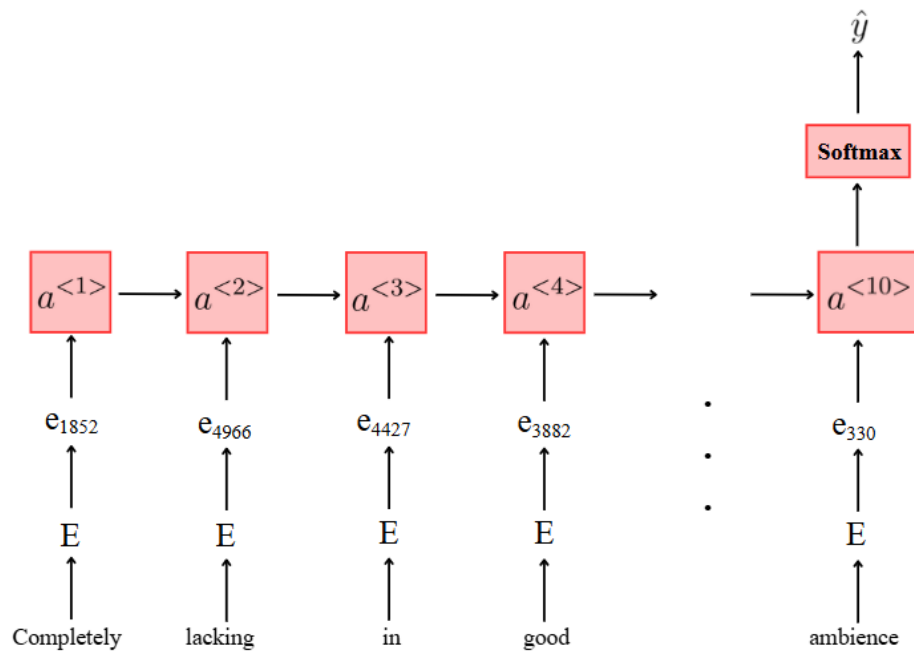
Example:



Note: Averaging is not the best idea as negative comments can also use good words, explained below

RNN for Sentiment Classification

Example:



The problem of bias in word embeddings

Example:

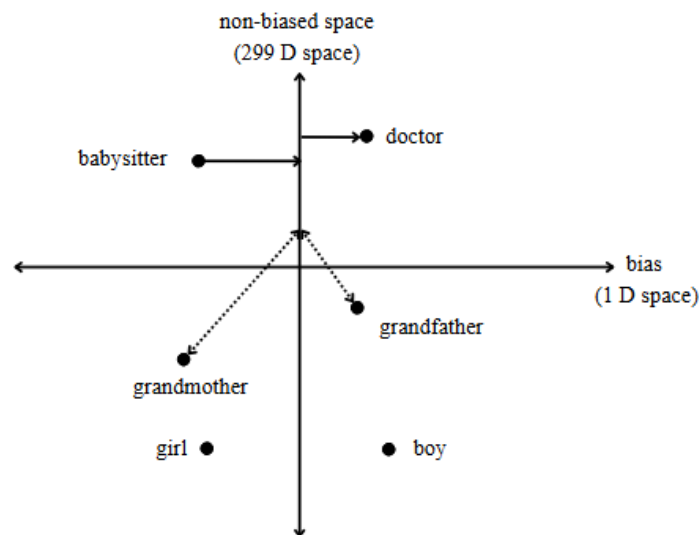
man : woman :: king : queen

man : computer-programmer :: woman : **home-maker**

Note: Stereotypes can be encouraged if the text used to train the model contained any stereotypical language/narrative.

Addressing Bias in word-embeddings

Example:



- 1. Identify bias direction

$$\text{average them} \leftarrow \begin{cases} e_{he} - e_{she} \\ e_{male} - e_{female} \\ \vdots \end{cases}$$

- 2. Neutralise: for every word that is not definitional, project to get rid of bias.
- 3. Equalise pairs